

Unit-3: List

Definition: A list is simply a collection of nodes to its successor. For a simple linear list, each node has a unique successor.

In other words, a list or sequence or contiguous list is a data structure that implements an ordered collection of values. Each value in the list is called an item, or an entry or element of the list.

-> It differs from stack and queue in such a way that additions and removals can be made at any position in the list.



Fig: Linear List

There are two types of lists:

- i. Static lists/Contiguous list/Array
- ii. Dynamic/linked List

i. Static List:

In a Static list, the space is allocated contiguously.

Advantages of static lists:

- > Allows random access to all the elements in the list.
- > Access time for each element in the list is constant.
- > Retrieval of the elements in the list is fast.

Disadvantages of Static List:

- > Memory needs to be allocated/reserved in advance. So, a fixed amount of storage remains always allocated even though it may not be fully occupied. There is a chance of overflow.
- > Changes like insertion and deletion of the elements is not efficient as it requires the shifting of elements.
- > Contiguous space should be available while allocating space.

ii. Dynamic List:

-> In a dynamic list, space is allocated dynamically.

Advantages of Dynamic List:

- > Provides flexibility. There is no need to make decisions for allocating storage in advance.
- > Overflow is not a problem until the computer memory is actually exhausted.
- > Changes, especially insertions and deletions, can be made in the middle of linked list much quickly as it requires changing the values of a few pointers.

Disadvantages of Dynamic List:

- > It is not suitable for random access.
- > It may be necessary to traverse a long path to reach the desired node.
- > Also, access to a single node may even take slightly more computer time. Since, it is necessary to obtain the pointer first and then go to the address.
- > As the links themselves take space, a list of elements(integers) will require double the space in linked storage that it would require in the contiguous list.

List Can be implemented in two ways:

- a. Contiguous implementation using arrays or Array Implementation of list
- b. Dynamic implementation of lists

a. Contiguous implementation using arrays or Array Implementation of list

-> In contiguous implementation, a list can be represented by an array of nodes in which each node carries within itself the location of its successor until the last node in the list, where next field contains -1, which is the null pointer.

-> Here, a structure is used for representing nodes of the list. The structure consists of two data items: info and next. The info consists the data on particular node, next consists the value for next node being represented.

-> The nodes of a list may be scattered throughout the arraynode in any arbitrary order.

-> So, a group of 100 nodes can be declared as:

```
#define NUMNODES 100
```

```
struct nodetype
```

```
{
```

```
    int info,next;
```

```
};
```

```
struct nodetype node[NUMNODES];
```

-> If next consists the value -1, then the node is the end of the list.

-> In this implementation, the pointer to a node is represented by an array index. It references a particular element in the array 'node'.

-> Suppose the variables 'head1' and 'head2' represent pointer to list1 and list2 respectively. Since, head1 has value 4, nodes[4] is the first node on the list1 and node[4].info is the first data item on the list1. The value of node[4].next is 8, so the second data item on the list1 is 45 and so on. If the next consists value (-1) i.e Null (eg: next[2].next in this case), the list is considered to be end. The list representation of list1 and list2 are shown below fig:

index no	Data	Nextnode
0	5	2
1	10	3
2	15	-1
3	20	6
4	25	8
5	30	0
6	35	-1
7	40	1
8	45	5

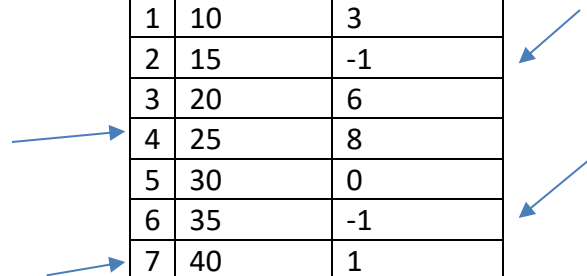


Fig. a. Array of nodes containing two linked lists.

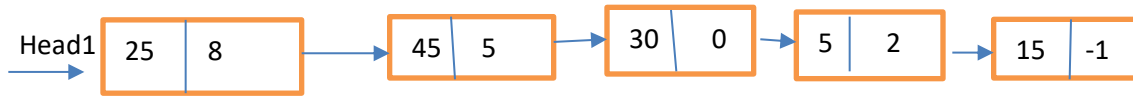


Fig. b. List1



Fig. c. List2

➔ The elements of all two lists are:

list1: head1

elements = 25, 45, 30, 5, 15

List2: head2

elements = 40, 10, 20, 35

➔ Initialization of the list elements can be done as follows:

/* initialization of list */

avail = 0

for(i=0; i<NUMNODES-1; i++)

```

{
    node[i].next = i+1;
}
  
```

node[NUMNODES-1].next = -1;

➔ The global variable avail is used to point the available list.

➔ In this case, its value 0 means the very first node the available list. All list nodes are ordered sequentially by node[i] pointing to the node[i+1]. The ordering of the node is explicit and hence can be ordered in any other way as well. i.e. instead of node[0] pointing to node[1] and so on. i.e. we can also point node[0] to node[24], node[24] to node[49] and so on.

The lastnode in our example is node[NUMNODES-1]; and so, it is set to -1 (i.e. Null) value.

Operations on list:

getnode: to create a new node.

freenode: destroys a node

insafter: insert an item into the list after a node

deafter: denotes the operation of deleting it.

➔ When a node is needed for use in a particular list, it is obtained from the available list. Similarly, it is not needed, then it is returned to the available list again.

Available is a pool of nodes conceptually.

➔ getnode is a function that removes a node from the available list. For insertion of new elements, a node is required which is provided by the following getnode function.

```

/* for getting the available node */
int getnode(void)
{
    int p;
    if(avail==-1)
    {
        printf("overflow");
        exit(1);
    }
    p=avail;
    avail=node[avail].next;
}

```

➔ Similarly, when the node is not required, it has to be returned to the list. The following function `freenode` does this job.

```

void freenode(int p)
{
    node[p].next=avail;
    avail=p;
    return;
}

```

Similarly, for inserting an item to node, the following function 'insertafter' is used which accepts a pointer `p` to a node and an item `x` as parameters. It first ensures that `p` is not null and then inserts `x` into a node following the node pointed to by `p`.

```

void insafter(int p, int x)
{
    int q;
    if(p==-1)
    {
        printf("void insertion\n");
        return;
    }
    q=getnode();
    node[q].info=x;
    node[q].next=node[p].next;
    node[p].next=q;
    return ;
}

```

➔ The function `delafter(p,*x)` deletes the node following `node(p)` and stores its contents in `x`:

```

/* for deleting a node and returning its content */
void delafter(int p, int *x)
{
    if(p== -1) || (node[p].next== -1)
    {
        printf("invalid deletion");
        exit(1);
    }
    q=node[p].next;
    *x=node[q].info;
    node[p].next=node[q].next;
    freenode(q);
}

```

Limitation of the array implementation

- ➔ First, the no. of nodes that are needed often cannot be predicted when a program is written. Usually, the data with which the program is executed determines the number of nodes necessary. Thus, no matter how many elements the array of nodes contains, it is always possible that the program will be executed with input that requires a larger number.
- ➔ The second disadvantage of the array approach is that whatever number of nodes are declared must remain allocated to the program throughout its execution. For eg: if 500 nodes of a given type are declared, the amount of storage required for those 500 nodes is reserved for that purpose. If the program actually uses only 100 or even 10 nodes in its execution, the additional nodes are still reserved and their storage cannot be used for any other purpose.

Note: The solution to this problem is to allow nodes that are dynamic, rather than static.

b. Dynamic implementation using pointers:

-> In the dynamic implementation, pointer variables are used to implement list pointers. So, the pointer and node can be defined as:

```

Struct node
{
    int info;
    struct node *next;
}

```

```
typedef struct node *NODEPTR;
```

- ➔ A node of this type is identical to the nodes of array implementation except for the content of the next field. For dynamic implementation, the next field is a pointer which contains the address of next node in the list. For array implementation, the next field contains the index (within an array) where the next node in the list is kept;
- ➔ The nodes are allocated and freed as necessary. In C, this can be accomplished by using the function 'malloc' and free respectively.

```

NODEPTR P;
P=(NODEPTR)malloc(sizeof(struct node));
free(P);

```

Linked List

- ➔ Linked list are special list of some data elements linked to one another. The logical ordering is represented by having each element pointing to the next element.
In other words, a linked list is a data structure in which each data item(or element) contains the address of the next data item within itself(or in the list).
- ➔ Each data item(or element) in the linked list is called a node and contains two fields, an information field and next address field.
- ➔ The information field contains the actual data which is to be stored in the list.
- ➔ The next address field contains the address of next node in the list.
- ➔ The entire list is pointed to by an external pointer called head, which contains the address of the first node and points to first node in the list. It is not part of linked list.
- ➔ The next address field of last node in the list contains a special value, known as null(or -1), which is not a valid address. This null pointer is used to signal the end of list.
- ➔ The list with no nodes on it is called the empty list or null list. The value of the external pointer head to such a list is the null pointer. eg: head=null.

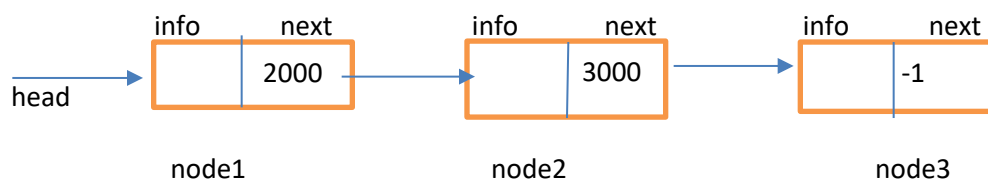


Fig: Linear linked list

- ➔ In C, a node for a linked list can be defined as:

```
struct linknode
{
    char name[30];
    int class;
    int rollNo;
    struct linknode *nextnode;
};
struct linknode *ptr;
ptr=(struct linknode *)malloc(sizeof(struct linknode));
```

Types of Linked List:

a. Singly-linked list:

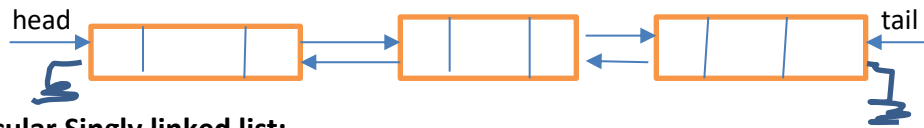
It is one in which all nodes are linked together in some sequential manner. Hence, it is also called linear linked list. Clearly, it has the beginning and the end. It is one way list where only the successor of any node can be accessed.



Fig: A singly linked list

b. Doubly linded list

It is one in which all nodes are linked together by multiple links which help in accessing both the successor node(next node) and predecessor node(previous node) for any arbitrary node within the list. For this purpose, each node is provided with two links, the left link(prev) and the right link(succ). This helps to traverse the list in forward and backward direction. It is a two-way list.



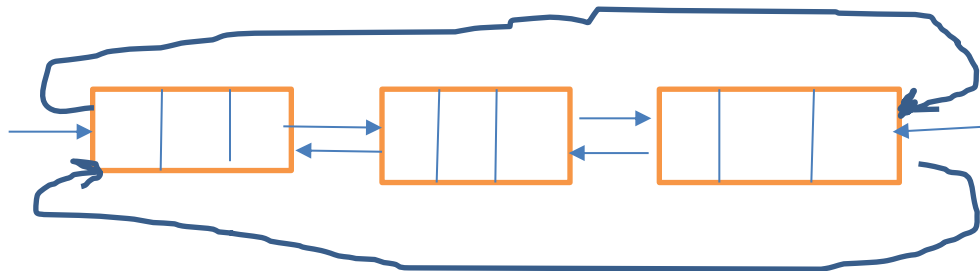
c. Circular Singly linked list:

It is one which has no beginning and no end. A singly linked list can be made a circular linked list by simply storing the address the address of the very first node in the link field of the lastnode.



d. Circular doubly linked list:

It is one which has both the successor pointer and predecessor pointer in circular manner.



Operations on linked list:

The basic operations to be performed on the linked list are as follows:

i. Creation:

This operation is used to create a linked list.

ii. Insertion:

This operation is used to insert a new node in the linked list at the specified position. A new node may be inserted:

- ➔ At the beginning of a linked list
- ➔ At the end of a linked list
- ➔ At the specified position in a linked list.
- ➔ If the list itself is empty, then the new node is inserted as a first node.

iii. Deletion:

This operation is use to delete an item(a node) from linked list. A node may be deleted from the

- > Begining of a linked list
- > End of a linked list
- > Specifed position in the list.

iv. Traversing:

It is a process of going through all the nodes of a lined list from one end. If we start traversing from the very first node toward the last node, it is called forward traversing. It may be forward or backward.

iv. Searching(or retrieval of data)

-> finds an element in the list.

v. **Concatenation:**

-> Appending (or joining) second list to the end of the first list consists of m nodes.

vii. **Destroy the linked list.**

Note: All these operations are similar to list and linked list.

i) **Creating a Singly linked list:**

```
struct linknode
{
    int data;
    struct a linknode *nextnode;
};
Struct linked * head;
headd=NULL;
```

ii. **Inserting a node:**

➔ The basic algorith is:

- a. Allocate space for the new node and then assign data.
- b. Identify the predecessor of the new node
- c. -> point the new node to its successor.
-> point the predecessor of new node.

a. **At the beginning of the singly linked list:**

- i. Declare *ptr of type struct linknode.
- ii. Allocate space for new node
ptr=(struct linknode *)malloc(sizeof(struct linknode));
- iii. If space is allocated successfully(i.e. ptr!=NULL) then continue else stop
- iv. Assign data
ptr->data =newdata
ptr->nextnode=NULL
- v. point newnode to its successor
ptr->nextnode=head
- vi. point(or link) new node's predecessor to new node.
head=ptr;
- b. At the end of the list:
 - i. Declare *ptr,*loc of type struct linknode.
 - ii. Allocate space for new space(or node).
ptr=(struct linknode *)malloc(sizeof(struct linknode));
 - iii. If space is allocated successfully(i.e ptr!=NULL) then continue else stop
 - iv. Assign data
ptr->data=newnode
ptr->nextnode=NULL
 - v. If the linked list is empty previously (i.e. head==NULL) then
 - a. head=ptr
 - b. stop(or exit) else continue
 - vi. Locate the last node in the list.
 - a. loc=head
 - b. while(loc->nextnode!=NULL)
loc=loc->nextnode

- vii. point the new node's predecessor to the new node.

loc->nextnode=ptr

c. At the middle of the singly linked list:

Algorithm:

- i. Declare *ptr and *loc of type struct linknode.
- ii. Allocate space for newnode
ptr=(struct linknode *) malloc(sizeof(struct linknode));
- iii. If space is allocated successfully(i.e ptr!=NULL)
then continue else stop
- iv. Assign data
ptr->data=newdata
ptr->nextnode=NULL
- v. Specify the node after which new node is to be inserted. Let the data of that node be "markdata".
- vi. Locate the node containing "markdata" in the list.
 - a. loc=head
 - b. while((loc->data!="markdata") &&(loc!=NULL))
loc=loc->nextnode

- vii. Point the new node to its successor

ptr->nextnode=loc->nextnode

- viii. Point new node's predecessor to the new node.

loc->nextnode=ptr

Inserting a newnode at the specified postion:

- i. Declare *ptr and *loc of type struct linknode.
- ii. Allocate space for newnode
ptr=(struct linknode *) malloc(sizeof(struct linknode));
- iii. If space is allocated successfully(i.e ptr!=NULL)
then continue else stop
- iv. Assign data
ptr->data=newdata
ptr->nextnode=NULL
- v. Specify the position of node after which new node is to be inserted. Let that position be "mark_pos".
- vi. Locate the node of that position in the list.
 - a. loc=head
 - b. initialize a counter n=1
 - c. while((mark_pos!=n)&&(loc!=NULL))
{
loc=loc->nextnode;
n++;
}
- v. Point the new node to its successor
ptr->nextnode=loc->nextnode
- vii. Point new node's predecessor to the new node.
loc->nextnode=ptr

Deleting the node from singly linked list

Basic guidelines

- i. Locate the node to be deleted and the address of its predecessor
- ii. Change the predecessor's link field to point to the successor of the deleted node.
- iii. free the memory of node.

Deletion of node at the beginning in the singly linked list

- i. Declare *delptr of type struct linknode
- ii. If the linked list is previously empty(i.e if (head==NULL) then
 - a. display "empty linked list
 - b. stop otherwise continue
- iii. Delete the node from linked list
 - a. delptr=head
 - b. head=head->nextnode
 - c. free(delptr)
 - d. stop

Deletion of node in a singly linked list at the end:

- i. Declare *delptr and *pred of type struct linknode.
- ii. If the linked list is previously empty(i.e if (head==NULL) then
 - c. display "empty linked list
 - d. stop otherwise continue
- iii. Locate the last node
 - a. delptr=head
 - b. while(delptr->nextnode!=NULL)
{
 pred=delptr;
 delptr=delptr->nextnode;
 }
 }
- iv. Delete the node from linked list
 - a. pred->nextnode=NULL
 - b. free(delptr)

Deletion of node in a singly linked list in the middle:

Algorithm:

- i. Declare *delptr and *pred of type struct linknode and 'pos' of type integer.
- ii. Ask the position of the node which is to be deleted. Let that position be 'mark_pos'.
- iii. if the mark_pos==1 (i.e the node to be deleted is the first node) then
 - a. delptr=head
 - b. head=head->nextnode
 - c. free(delptr)
 - d. stop otherwise continue
- iv. Locate the node to be deleted.
 - a. delptr=head
 - b. pos=1
 - c. while((pos!=markpos)&&(delptr!=NULL))
{
 pred=delptr;

- ```

 delptr=delptr->nextnode;
 pos++;
 }

```
- v. If the specified node doesn't exist(i.e if(delptr==NULL),then
    - a. display"specified position doesn't exist
    - b. stop else continue
  - vi. Delete the node
    - a. pred->nextnode=delptr->nextnode
    - b. free(delptr)
    - c. stop

### Traversing the linked list:

This algorithm traverses the linked list and then displays the data of each node.

Algorithm:

- i. Declare \* ptr of type struct linknode
- ii. if the linked list contains a single node[i.e head->nextnode==NULL) then
  - a. display head->data
  - b. stop else continue
- iii. a. ptr=head
  - b. while(ptr!=NULL)
 

```

 {
 display:ptr->data;
 ptr=ptr->nextnode
 }

```
  - c. stop

### Doubly Linked lists:

- ➔ A doubly linked list is a data structure in which each node contains two pointers, one to its predecessor and another to its successor.
- ➔ It provides bi-directional traversing.
- ➔ The nodes in a doubly linked list consists of three fields:
  - a. an information field which contains the actual information to be stored in that node.
  - b. Left address field which contains the address of the node's predecessor.
  - c. Right address field which contains the address of node's successor.
- ➔ Two pointer variables, head and tail, are used to keep track of the list elements. One refers to the first node while other refers to the last node.
- ➔ The first node of the list has no predecessor. Therefore, its reference is NULL.
- ➔ Similarly, the last node of the list has no successor. So, its right address field points to NULL.

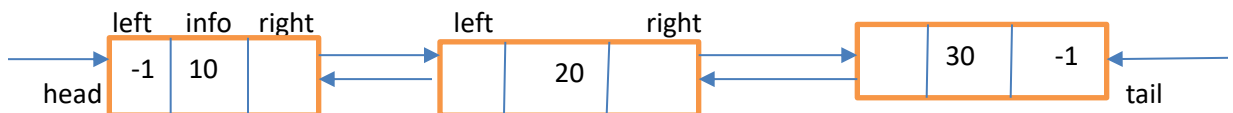


fig: A linear doubly linked list

➔ Creation of a double linked list:

```
struct doublenode
{
 int data;
 struct doublenode *right;
 struct doublenode *left;
};

struct doublenode *head,*tail;
head=tail=NULL;
```

## Insertion of a node in a doubly linked list;

### a. At the beginning:

- i. Declare \*ptr of type struct doublenode.
- ii. Allocate space  
ptr=(struct doublenode \*)malloc(sizeof(struct doublenode));
- iii. If space is allocated successfully(i.e ptr!=NULL) then continue else stop
- iv. Assign data  
ptr->data=newdata  
ptr->left=NULL  
ptr->right=NULL
- v. Insert the node at the beginning of the list
  - a. ptr->right=head
  - b. head->left=ptr
  - c. head=ptr
- vi. stop

### b. At the end of list

- i. Declare \*ptr of type struct doublenode.
- ii. Allocate space for newnode i.e  
ptr=(struct doublenode \*)malloc(sizeof(struct doublenode));
- iii. If space is allocated successfully(i.e ptr!=NULL) then continue else stop
- iv. Assign data  
ptr->data=newdata  
ptr->left=NULL  
ptr->right=NULL
- v. insert the node at the end of list
  - a. tail->right=ptr
  - b. ptr->left=tail
  - c. tail=ptr
- vi. stop

### **Insertion of a node after a given node**

- i. Declare \*ptr,\*loc of type struct doublenode.
- ii. Specify the data of the node after which the new node is to be inserted. Let the data of that node be 'markdata'.
- iii. Locate the specified node
  - a. loc=head
  - b. while((loc->data!=markdata) &&(loc!=NULL))  
{ loc=loc->right;  
}
- iv. Allocate space for newnode  
ptr=(struct doublenode \*)malloc(sizeof(struct doublenode));
- v. If space is allocated successfully(i.e ptr!=NULL) then continue else stop
- vi. Assign data  
ptr->data=newdata  
ptr->left=NULL  
ptr->right=NULL
- vii. Insert the node after the specified node
  - a. (loc->right)->left=ptr
  - b. ptr->right=loc->right
  - c. ptr->left=loc
  - d. loc->right=ptr
- viii. stop

### **Insertion of a node before a given node**

- i. Declare \*ptr,\*loc of type struct doublenode.
- ii. Specify the data of the node after which the new node is to be inserted. Let the data of that node be 'markdata'.
- iii. Locate the specified node
  - a. loc=head
  - b. while((loc->data!=markdata) &&(loc!=NULL))  
{ loc=loc->right;  
}
- iv. Allocate space for newnode  
ptr=(struct doublenode \*)malloc(sizeof(struct doublenode));
- v. If space is allocated successfully(i.e ptr!=NULL) then continue else stop
- vi. Assign data  
ptr->data=newdata  
ptr->left=NULL  
ptr->right=NULL
- vii. Insert the node before the specified node
  - a. (loc->left)->right=ptr
  - b. ptr->left=loc->left
  - c. ptr->right=loc
  - d. loc->left=ptr
- viii. stop

### **Deleting a node from the doubly linked list**

- a. Deleting a node from the beginning:
  - i. Declare \*delptr of type struct doublenode.
  - ii. If the linked list is previously empty(i.e(head=tail=NULL) then
    - a. display “empty linked list”
    - b. stop else continue
  - iii. If second node is not null) (i.e.(head->right!=NULL) then
    - a. delptr=head
    - b. head=head->right
    - c. head->left=NULL
    - d. free(delptr)

### **Deleting a node from the end of doubly linked list**

- b. Deleting a node from the beginning:
  - iv. Declare \*delptr of type struct doublenode.
  - v. If the linked list is previously empty(i.e(head=tail=NULL) then
    - c. display “empty linked list”
    - d. stop else continue
  - vi. If second node is not null) (i.e.(tail->left!=NULL) then
    - e. delptr=tail
    - f. tail=tail->left
    - g. tail->right=NULL
    - h. free(delptr)
  - vii. Stop

### **Deletion of a node in doubly linked list in the middle**

- i. Declare \*delptr of type struct doublenode and pos of type integer.
- ii. Ask the position of the node which is to be deleted. Let that position be markpos.
- iv. Locate the node to be deleted.
  - a. delptr=head
  - b. pos=1
  - c. while(pos!=markpos && delptr!=NULL)

```
{
 delptr=delptr->right;
 pos++;
}
```
- v. Delete the node where position=markpos
  - a. delptr->left->right=delptr->right
  - b. delptr->right->left=delptr->left
  - c. free(delptr)
  - d. stop

## Linked Stacks and Queues

### Linked implementation Stacks or linked Stacks:

- ➔ A linked list can be manipulated to implement the stack. In a linked stack, header variables (i.e. head now stacktop) always points to the firstnode of the list.
- ➔ A stack may be represented by a linear linked list. The firstnode of the list is the top of the stack.
- ➔ Structure of linked stack node is:

```
struct stacknode
{
 int data;
 struct stacknode *nextnode;
};
Struct stacknode *stacktop;
stacktop=NULL;
```

### Push Algorithm

- i. *Declare* a pointer variable \*ptr of type struct stacknode
- ii. Allocate space  
ptr=(struct stacknode \*)malloc(sizeof(stacknode));
- iii. If space is allocated successfully(i.e. ptr!=NULL) then continue else stop
- iv. Assign data  
ptr->data=newdata  
ptr->nextnode=NULL
- v. if the linked stack is previously empty(i.e if(stacktop ==NULL) then
  - a. stacktop=ptr
  - b. stop else continue
- vi. insert node at the begining
  - a. ptr->nextnode=stacktop
  - b. stacktop=ptr
- vii. stop

### POP Algorithm:

- i. Declare \*delptr of type struct stacknode
- ii. If the linked stack is previously empty(i.e stacktop==NULL) then
  - a. display "empty"
  - b. stop otherwise continue
- iii. Remove the first node from the linked stack
  - a. delptr=stacktop
  - b. stacktop=stacktop->nextnode
  - c. free(delptr)
- v. stop

## Linked Queue or linked implementation of Queue

- ➔ It is an alternative implementation of queue
- ➔ In contiguous storage, queues were significantly harder to manipulate than stacks.
- ➔ Linked queues are much easier to handle than linked stacks. We need to keep two pointers 'front' and 'rear' that will point to the beginning and end of the linked queue respectively.
- ➔ Structure of linked queue node:

```
struct queueNode
{
 int data;
 struct queueNode *nextNode;
};
struct queueNode *front,*rear;
front=NULL,rear=NULL;
```

### Enqueue Algorithm:

- i. Declare \*ptr of type struct queueNode
- ii. Allocate space  
ptr=(struct queueNode \*)malloc(sizeof(struct queueNode))
- iii. If space is successfully allocated (i.e ptr!=NULL) then continue else stop
- iv. Assign data  
ptr->data=newdata  
ptr->nextNode=NULL
- v. If the queue is previously empty (i.e (rear==front==NULL) then
  - a. front=ptr
  - b. rear=ptr
  - c. stop otherwise continue
- vi. insert the node at the rear end
  - a. rear->nextNode=ptr
  - b. rear=ptr

### Deque Algorithm

- i. Declare \*delPtr of type struct queueNode
- ii. If the queue is previously empty (i.e (rear==front)==NULL) then
  - a. display "empty linked queue"
  - b. stop otherwise continue
- iii. if the linked queue contains only a single node (i.e (rear==front)!=NULL) then
  - a. delPtr=front
  - b. front=NULL
  - c. rear=NULL
  - d. free(delPtr)
  - e. stop otherwise continue
- iv. Remove the front end
  - a. delPtr=front
  - b. front=front->nextNode
  - c. free(delPtr)
- v. Stop



